

SmartSafe Master Unit

Complete Design and Code Explanation of the ADC Tamper-Detection Module

Microprocessors End-Term Project

July 2026

Abstract

This report documents the SmartSafe tamper-detection subsystem. A potentiometer connected to PA3/ADC3 simulates displacement, vibration, or impact. The ATmega32 ADC converts the analog wiper voltage into a 10-bit sample approximately every 100 ms. The report explains the sensor wiring, why the pin is an input with its internal pull-up disabled, the ADMUX and ADCSRA register settings, AVCC reference selection, ADC clock and conversion-time calculations, voltage resolution, the choice of a 50-count threshold, Timer2 CTC calculations, the difference between consecutive-sample and stable-reference detection, event rearming, main-loop integration, USART notification and the 9600-baud calculation, external EEPROM logging, sleep and power-restoration behavior, and detailed Proteus testing and troubleshooting.

Contents

1 Purpose of the Tamper Subsystem	3
2 Hardware Connection	3
3 Why the Internal Pull-Up Is Disabled	3
4 ADC Register Configuration	4
4.1 ADMUX configuration	4
4.2 ADCSRA configuration	4
5 ADC Clock Calculation	5
6 Channel Selection and Conversion Procedure	5
7 ADC Resolution and Threshold Calculation	6
8 Periodic Sampling with Timer2	6
9 Detection Algorithms	6
9.1 Earlier consecutive-sample method	6
9.2 Recommended stable-reference method	7
10 Initialization and Baseline Selection	7
11 Arming and Rearming	8
12 Main-Loop Integration	8
13 USART Notification and Baud-Rate Calculation	8

14 Event Logging	9
15 Master and Slave Responsibilities	9
16 Interaction with Sleep and Power Failure	9
16.1 Normal inactivity Power-down	9
16.2 Power-failure standby	10
17 Timing and Constant Summary	10
18 Proteus Test Procedure	10
19 Common Failure Modes	11
20 Conclusion	11

1 Purpose of the Tamper Subsystem

The project specification requires periodic monitoring of a simulated vibration or impact sensor and an alarm when the sensor changes suddenly beyond a threshold. In Proteus, a potentiometer represents this sensor. The Master performs detection and sends an event to the independently powered Slave. The Slave owns the red LED, buzzer driver, and alarm LCD message.

A new tamper event causes the Master to:

1. transmit `Tamper\r\n` through USART;
2. store `EVT_TAMPER` with a timestamp in the external SPI EEPROM;
3. block keypad input for five seconds without replacing the Master password prompt;
4. reset the inactivity counter so the Master does not immediately sleep.

2 Hardware Connection

The potentiometer is connected as a voltage divider:

Potentiometer terminal	Connection
Upper end	+5 V
Lower end	GND
Wiper	PA3/ADC3

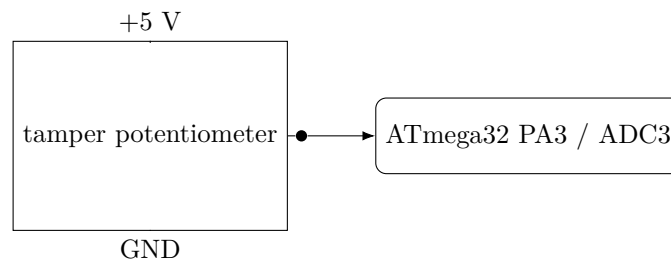


Figure 1: Tamper potentiometer and ADC3 connection.

The analog supply requirements are:

- AVCC connected to +5 V;
- both MCU grounds connected to GND;
- AREF bypassed to ground with approximately 100 nF when AVCC is used as the reference;
- no direct short from AREF to ground;
- the potentiometer and MCU share the same ground.

3 Why the Internal Pull-Up Is Disabled

The ADC pin is configured as an input and its pull-up is disabled:

```

1 ADC_TAMPER_DDR  &= ~(1U << ADC_TAMPER_PIN);
2 ADC_TAMPER_PORT &= ~(1U << ADC_TAMPER_PIN);

```

For an AVR pin, `DDRx=0` selects input. With input selected, `PORTx=1` would connect a weak internal resistor to VCC. That resistor would become part of the external potentiometer network, pull the wiper voltage upward, and distort the ADC result. The potentiometer already gives the pin a defined analog voltage, so no digital bias resistor is needed.

This is different from keypad columns, where an internal pull-up is intentionally used to define the idle state of an otherwise disconnected digital switch input.

4 ADC Register Configuration

The initialization code is:

```

1 void adc_init(void)
2 {
3     ADC_TAMPER_DDR  &= ~(1U << ADC_TAMPER_PIN);
4     ADC_TAMPER_PORT &= ~(1U << ADC_TAMPER_PIN);
5
6     ADMUX  = (1U << REFS0);
7     ADCSRA = (1U << ADEN) |
8               (1U << ADPS2) |
9               (1U << ADPS1);
10 }
```

Listing 1: ADC initialization for the tamper sensor.

4.1 ADMUX configuration

The relevant fields are:

Field	Value	Result
REFS1:REFS0	01	AVCC used as ADC reference
ADLAR	0	Conversion result right-adjusted
MUX4:0 at initialization	00000	Channel selected later by <code>adc_read()</code>

With `REFS0=1` and `REFS1=0`, the conversion range is nominally 0 V to AVCC. The AREF pin is used for decoupling the selected internal reference path.

Right adjustment allows the complete 10-bit result to be read conveniently through the AVR-GCC ADC 16-bit register macro.

4.2 ADCSRA configuration

Bit	Value	Purpose
ADEN	1	Enable the ADC peripheral.
ADSC	0 at initialization	A conversion is started later by software.
ADATE	0	Disable auto-trigger; use single conversions.
ADIF	0	Conversion-complete flag not used for interrupt operation.
ADIE	0	ADC interrupt disabled; the driver polls completion.
ADPS2:0	110	Prescaler 64.

5 ADC Clock Calculation

The CPU clock is 8 MHz. With prescaler 64:

$$f_{ADC} = \frac{8\,000\,000}{64} = 125\,000 \text{ Hz} = 125 \text{ kHz}.$$

One ADC clock period is

$$T_{ADC} = \frac{1}{125\,000} = 8 \mu\text{s}.$$

A normal single conversion requires approximately 13 ADC clock cycles, so its approximate conversion time is

$$13 \times 8 = 104 \mu\text{s}.$$

The first conversion after enabling the ADC requires additional initialization cycles and is longer, approximately 25 ADC clocks or 200 microseconds. This is still much shorter than the 100 ms sampling interval.

The 125 kHz ADC clock is chosen because it is within the range conventionally recommended for full 10-bit accuracy on classic AVR devices, while remaining fast enough for this low-frequency sensor.

6 Channel Selection and Conversion Procedure

The read function is:

```

1 uint16_t adc_read(uint8_t channel)
2 {
3     ADMUX = (ADMUX & 0xE0U) | (channel & 0x07U);
4
5     ADCSRA |= (1U << ADSC);
6     while (ADCSRA & (1U << ADSC)) {
7         /* wait for hardware to clear ADSC */
8     }
9
10    return ADC;
11 }
```

Listing 2: Single-channel ADC read.

For the tamper sensor, `channel=3`, so the multiplexer bits become 00011, selecting PA3/ADC3. The mask preserves the reference-selection and result-alignment bits.

The sequence is:

1. select ADC3 in ADMUX;
2. set ADSC to start conversion;
3. wait while ADSC remains one;
4. hardware clears ADSC when conversion finishes;
5. read the 10-bit result.

On the 8-bit AVR bus, the low result byte must be read before the high byte. The compiler-provided ADC macro performs the correct ordered access for a 16-bit result.

7 ADC Resolution and Threshold Calculation

For a 10-bit ADC, output codes range from 0 through 1023. With a nominal 5 V reference, one least-significant bit represents approximately

$$V_{LSB} = \frac{5}{1024} = 4.8828 \text{ mV}.$$

The selected threshold is:

```
1 #define TAMPER_THRESHOLD 50U
```

Its voltage equivalent is approximately

$$50 \times 4.8828 \text{ mV} = 244.14 \text{ mV}.$$

A 50-count change is large enough to reject minor quantization noise and small simulation fluctuations, yet small enough to respond to deliberate potentiometer movement.

If AVCC differs from exactly 5 V, the voltage represented by one count changes proportionally. The algorithm is fundamentally based on ADC counts, so its effective voltage threshold tracks the reference voltage.

8 Periodic Sampling with Timer2

The ADC is sampled when Timer2 produces a 100 ms software flag. Timer2 is configured in CTC mode with:

```
1 OCR2 = 77U;
2 TIMSK |= (1U << OCIE2);
3 TCCR2 = (1U << WGM21) |
4         (1U << CS22) |
5         (1U << CS21) |
6         (1U << CS20);
```

The prescaler is 1024 and the compare period is

$$T_{match} = \frac{1024(77 + 1)}{8\,000\,000} = 9.984 \text{ ms}.$$

Ten matches produce approximately 99.84 ms. The ISR sets `timer_tick_100ms=1`; the main loop clears it and performs one ADC sample.

The ADC conversion itself takes only about 104 microseconds, approximately 0.1 percent of the 100 ms interval. Therefore periodic tamper sampling has negligible CPU cost.

9 Detection Algorithms

9.1 Earlier consecutive-sample method

A simple implementation calculates

$$difference = |sample_n - sample_{n-1}|.$$

It works well for a sudden step. However, gradual manual movement can be missed. For example:

$$500, 510, 520, 530, 540, 550$$

has a total movement of 50 counts, but every consecutive difference is only 10 counts.

9.2 Recommended stable-reference method

The final recommended method compares the current sample with a stable reference that is not replaced during ordinary armed monitoring. Several small movements therefore accumulate relative to the baseline.

```

1  #define TAMPER_REARM_BAND      5U
2  #define TAMPER_STABLE_SAMPLES  3U
3
4  static uint16_t reference_adc = 0U;
5  static uint8_t armed = 1U;
6  static uint8_t stable_samples = 0U;
7
8  static uint16_t adc_difference(uint16_t a, uint16_t b)
9  {
10     return (a >= b) ? (a - b) : (b - a);
11 }
12
13 void tamper_init(void)
14 {
15     reference_adc = adc_read(ADC_TAMPER_CH);
16     armed = 1U;
17     stable_samples = 0U;
18 }
19
20 uint8_t tamper_check(void)
21 {
22     uint16_t value = adc_read(ADC_TAMPER_CH);
23     uint16_t difference = adc_difference(value, reference_adc);
24
25     if (armed) {
26         if (difference > TAMPER_THRESHOLD) {
27             armed = 0U;
28             stable_samples = 0U;
29             reference_adc = value;
30             return 1U;
31         }
32         return 0U;
33     }
34
35     if (difference <= TAMPER_REARM_BAND) {
36         if (++stable_samples >= TAMPER_STABLE_SAMPLES) {
37             armed = 1U;
38             stable_samples = 0U;
39         }
40     } else {
41         reference_adc = value;
42         stable_samples = 0U;
43     }
44
45     return 0U;
46 }

```

Listing 3: Robust stable-reference tamper detector.

10 Initialization and Baseline Selection

`tamper_init()` performs an ADC conversion and uses the present potentiometer position as the initial reference. This prevents startup from being interpreted as tampering merely because the

source code chose a fixed default such as 512 while the actual potentiometer starts elsewhere.

The same initialization is performed after power-restoration reinitialization. The new stable physical position becomes the baseline after the Master returns from power-failure standby.

11 Arming and Rearming

After a threshold crossing, the detector returns exactly one event and becomes disarmed. Without this state, a displaced potentiometer could generate a new event every 100 ms.

The rearm logic requires three consecutive readings within five ADC counts of the current reference:

$$5 \text{ counts} \times 4.8828 \text{ mV} \approx 24.4 \text{ mV}.$$

Three 100 ms samples require approximately 300 ms of stability. This gives the simulated sensor time to settle before a new event can be accepted.

When the sensor continues moving while disarmed, the reference follows the latest value and the stability counter restarts. Once motion stops, the stable-sample requirement is satisfied and the detector rearms at the new resting position.

12 Main-Loop Integration

The relevant main-loop structure is:

```

1  if (timer_tick_100ms) {
2      uint32_t now;
3      timer_tick_100ms = 0U;
4
5      if (tamper_check()) {
6          now = timer_get_seconds();
7          usart_print(MSG_TAMPER);
8          log_event(EVT_TAMPER, now);
9
10         ui_hold_until = now + TAMPER_UI_HOLD_SEC;
11         activity_timer = 0U;
12     }
13 }
```

The low-level detector only returns a Boolean event. It does not call the LCD, USART, logger, or alarm outputs. This separation keeps the sensor module reusable and keeps system policy in `main.c`.

13 USART Notification and Baud-Rate Calculation

The tamper message is:

```

1  #define MSG_TAMPER "Tamper\r\n"
```

The project uses normal asynchronous USART mode at 9600 baud with an 8 MHz CPU clock. The UBRR equation is

$$UBRR = \frac{F_{CPU}}{16 \times Baud} - 1.$$

Therefore

$$UBRR = \frac{8\,000\,000}{16 \times 9600} - 1 = 51.0833.$$

The nearest integer is 51. The actual baud rate is

$$Baud_{actual} = \frac{8\,000\,000}{16(51 + 1)} = 9615.38 \text{ baud.}$$

The error is

$$\frac{9615.38 - 9600}{9600} \times 100 \approx 0.16 \text{ percent.}$$

The USART configuration is:

```

1 UBRRH = 0U;
2 UBRRL = 51U;
3 UCSRB = (1U << RXEN) | (1U << TXEN);
4 UCSRC = (1U << URSEL) |
5         (1U << UCSZ1) |
6         (1U << UCSZ0); /* 8 data, no parity, 1 stop */

```

The string `Tamper\r\n` contains eight characters. In 8N1 format each character occupies ten serial bits: one start, eight data, and one stop. Its approximate transmission time is therefore

$$\frac{8 \times 10}{9600} = 8.33 \text{ ms.}$$

This is much shorter than the 100 ms sensor interval and is adequate for remote alarm notification.

14 Event Logging

After transmission, the Master stores `EVT_TAMPER` and the current seconds timestamp in the 25LC040 SPI EEPROM. Logging is performed outside the ADC routine so that analog acquisition remains independent from the storage protocol. The external EEPROM record survives reset and can later be dumped through the virtual terminal.

The logger and USART operations may take several milliseconds, but the detector is disarmed immediately after the event. Therefore the same disturbance does not produce repeated entries while the notification is being processed.

15 Master and Slave Responsibilities

Unit	Responsibility
Master	Sample ADC3, evaluate threshold, timestamp and log event, send Tamper , block keypad temporarily.
Slave	Match the received message, display the tamper warning, activate the red LED and buzzer driver for the required duration.

The Master has no local alarm LED or buzzer in the final design.

16 Interaction with Sleep and Power Failure

16.1 Normal inactivity Power-down

When the Master enters inactivity Power-down, Timer2 and the ADC stop. Moving the tamper potentiometer cannot wake the Master. The INT0 wake button must first wake normal operation, after which periodic ADC sampling resumes.

16.2 Power-failure standby

After a confirmed monitored power failure, the Master disables the ADC and Timer2. Only the analog comparator remains as the restoration wake source. After restoration, the runtime initialization sequence reenables ADC3, reads a new tamper baseline, restarts Timer2, and resumes monitoring.

17 Timing and Constant Summary

Item	Value	Reason
CPU clock	8 MHz	Common project clock and convenient peripheral calculations.
ADC reference	AVCC, nominal 5 V	Full potentiometer range corresponds to ADC codes 0–1023.
ADC prescaler	64	Produces 125 kHz ADC clock.
Normal conversion time	about 104 microseconds	13 ADC clocks at 125 kHz.
ADC resolution	about 4.883 mV/-count	5 V divided by 1024.
Tamper threshold	50 counts	Approximately 244 mV; rejects small noise.
Timer2 prescaler	1024	Allows an approximately 10 ms compare interval with an 8-bit timer.
OCR2	77	78 counts produce 9.984 ms.
Sampling interval	about 99.84 ms	Ten Timer2 compare matches.
Rearm band	5 counts	Approximately 24.4 mV stability band.
Stable samples	3	Approximately 300 ms before rearming.
USART UBRR	51	Actual baud approximately 9615, error 0.16 percent.
Tamper UI hold	5 s	Matches the ordinary remote alarm interval.

18 Proteus Test Procedure

1. Set the Master ATmega32 clock property to 8 MHz.
2. Connect the potentiometer ends to +5 V and GND and the wiper to PA3/ADC3.
3. Connect AVCC to +5 V and place a capacitor from AREF to ground.
4. Wake the Master and confirm the password prompt is active.
5. Note the initial potentiometer position; it becomes the baseline.
6. Move the wiper enough to exceed approximately 50 ADC counts.
7. Confirm the Master sends **Tamper** and stores one event.
8. Confirm the Slave displays the tamper status and activates its alarm.
9. Hold the potentiometer at the new position; no repeated event should occur.
10. Keep it stable for at least about 300 ms, then move it again to test rearming.

For diagnosis, the ADC result can be temporarily printed to the terminal or watched in the debugger. At 0 V it should be near 0, near half supply it should be around 512, and near 5 V it should approach 1023.

19 Common Failure Modes

Symptom	Likely cause and correction
ADC always reads zero	AVCC is not powered, AREF is incorrectly shorted, the wiper is not connected to PA3, or ADC is disabled.
ADC always reads near 1023	Pull-up is enabled on PA3, wiper is tied to VCC, or lower potentiometer terminal is not grounded.
Slow movement does not trigger	Consecutive-sample algorithm is still used; use the stable-reference algorithm.
Repeated events every 100 ms	Detector is not disarmed after the first event or rearm logic is missing.
No event after ten seconds	Master entered Power-down; wake it with INT0 before testing.
Tamper works on Master but not Slave	USART wiring, common ground, 9600 8N1 settings, or exact message matching is wrong.
False event immediately at startup	Fixed baseline was used instead of reading the actual startup ADC value.
Tamper stops after power restoration	ADC, Timer2, or <code>tamper_init()</code> was not reinitialized.

20 Conclusion

The tamper subsystem combines a simple analog sensor simulation with deterministic digital processing. ADC3 is configured with AVCC reference, right-adjusted 10-bit results, and a /64 prescaler that produces a 125 kHz ADC clock. Timer2 schedules a sample every approximately 100 ms. A 50-count threshold corresponds to about 244 mV at a 5 V reference. The stable-reference and rearm state machine detects both sudden and accumulated displacement while preventing duplicate alarms. USART then transports the event at 9600 baud and the external EEPROM preserves it for later inspection.

References

- [1] Microchip Technology Inc., *ATmega32A Data Sheet*, sections on ADC, Timer/Counter2, USART, I/O ports, and sleep modes.
[Official Microchip PDF](#)
- [2] Microchip Technology Inc., *AVR042: AVR Hardware Design Considerations*, analog supply, reference, and decoupling guidance.
[Official Microchip application note](#)
- [3] Microprocessors End-Term Project, *Smart Safe with Remote Alarm*, project specification supplied with the implementation.
- [4] SmartSafe source files: `adc.c`, `tamper.c`, `timer.c`, `usart.c`, `config.h`, and `main.c`, July 2026 project version.